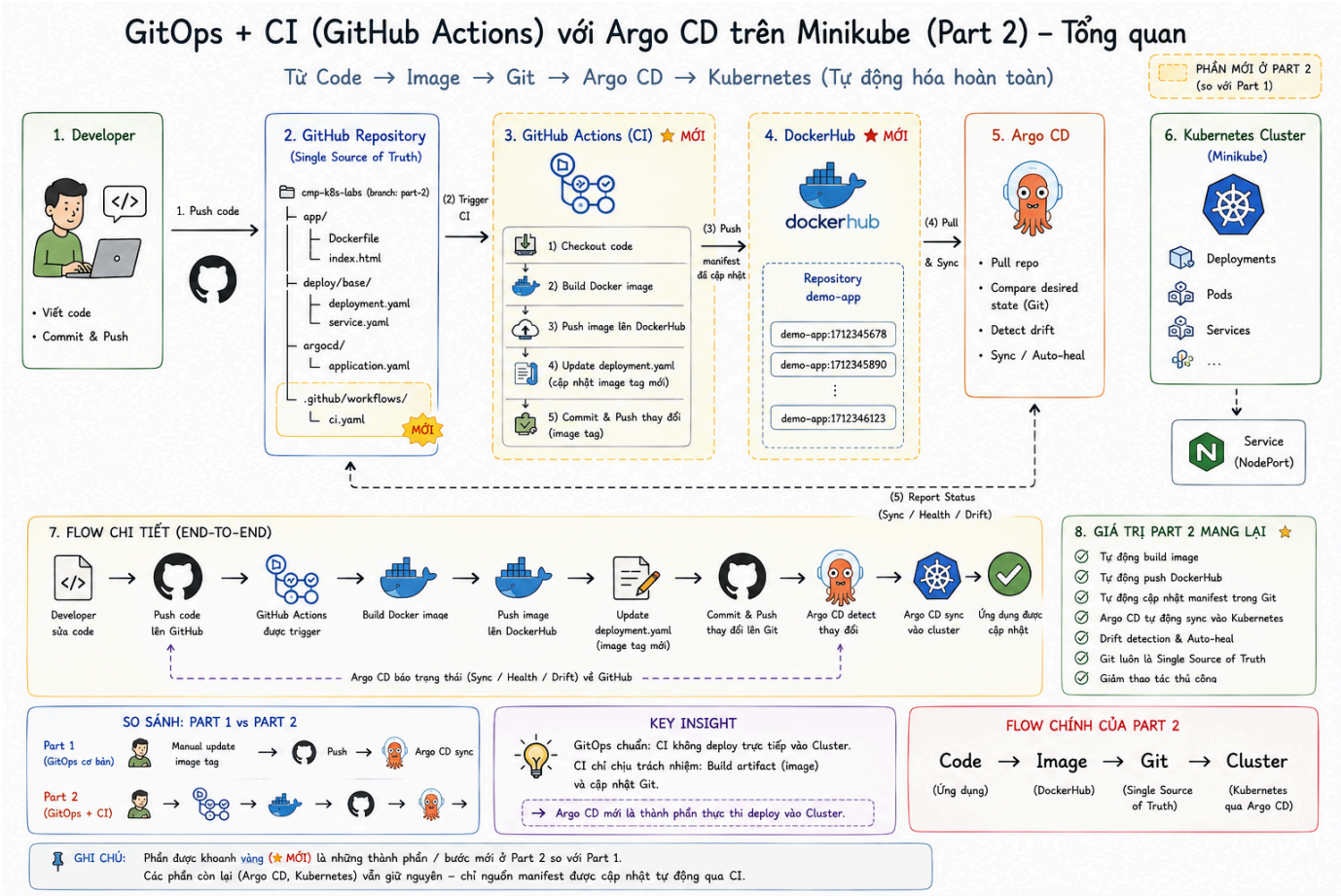
Part 2 — Xây dựng CI an toàn cho GitOps

> Thời lượng dự kiến: 75–100 phút Kết quả: GitHub Actions build một image truy vết được tới commit, cập nhật manifest một lần và Argo CD rollout phiên bản mới mà không tạo CI loop.



1. Vấn đề cần giải quyết

Ở Part 1, image tag được sửa thủ công. Part này tự động hóa flow:

```
Developer thay đổi app/
          ↓
GitHub Actions: build → push image → cập nhật manifest
          ↓
Git chứa desired state mới
          ↓
Argo CD reconcile cluster
```

CI không cần credential của cluster và không chạy `kubectl apply`. Trong thiết kế pull-based của series, CI tạo artifact và cập nhật desired state; Argo CD chịu trách nhiệm reconcile.

2. Mục tiêu và liên hệ CKA/CKAD

Sau bài này, bạn có thể:

- Build và push container image bằng GitHub Actions.
- Dùng commit SHA làm immutable tag để truy vết source → image → workload.
- Cập nhật đúng một key trong YAML và tránh workflow tự kích hoạt vô hạn.
- Kiểm tra rollout, image đang chạy và nội dung thực tế của app.
- Chẩn đoán `ImagePullBackOff` bằng `describe` và Events.

Nội dung	CKAD	CKA	Kỹ năng thực tế
Image, Deployment rollout, logs/events	Trọng tâm	Liên quan	Có
GitHub Actions, Docker Hub	Không trực tiếp	Không trực tiếp	Trọng tâm
GitOps deployment flow	Không trực tiếp	Không trực tiếp	Trọng tâm

3. Điều kiện chuẩn bị

- Đã hoàn thành Part 1; `demo-app` đang `Synced/Healthy`.
- Docker Hub account, repository public tên `demo-app`, và access token có quyền push. Không dùng password chính của tài khoản.
- Repository GitHub của bạn cho phép workflow ghi nội dung. Nếu branch có protection, dùng pull request thay vì push trực tiếp hoặc điều chỉnh rule cho lab.
- Các command Git dưới đây chạy trên Bash/Git Bash/WSL.

Tạo branch từ trạng thái hoàn tất Part 1:

```
git checkout part-1
git pull
git checkout -b part-2
```

4. Cấu trúc repository

```
cmp-k8s-labs/
├── app/
│   ├── Dockerfile
│   ├── .dockerignore
│   ├── index.html
│   └── nginx.conf
├── deploy/base/
│   ├── namespace.yaml
│   ├── deployment.yaml
│   └── service.yaml
├── argocd/application.yaml
└── .github/workflows/ci.yaml
```

5. Thực hành

Bước 1: Tạo ứng dụng và container image

`app/index.html`:

```
<!doctype html>
<html lang="vi">
  <head><meta charset="utf-8"><title>GitOps demo</title></head>
  <body><h1>Hello from CI Pipeline v1</h1></body>
</html>
```

`app/nginx.conf` (main config cho phép nginx chạy non-root và chỉ ghi file tạm vào `/tmp`):

```
pid /tmp/nginx.pid;
error_log /dev/stderr warn;

events {}

http {
  include /etc/nginx/mime.types;
  access_log /dev/stdout;
  client_body_temp_path /tmp/client_temp;
  proxy_temp_path /tmp/proxy_temp;
  fastcgi_temp_path /tmp/fastcgi_temp;
  uwsgi_temp_path /tmp/uwsgi_temp;
  scgi_temp_path /tmp/scgi_temp;

  server {
    listen 8080;
    server_name _;
    root /usr/share/nginx/html;
    location / { try_files $uri $uri/ =404; }
  }
}
```

`app/Dockerfile`:

```
FROM nginx:1.26-alpine
COPY --chown=nginx:nginx index.html /usr/share/nginx/html/index.html
COPY nginx.conf /etc/nginx/nginx.conf
USER nginx
EXPOSE 8080
```

`app/.dockerignore`:

```
*.md
.DS_Store
```

Docker build context của workflow là `./app`, vì vậy `.dockerignore` phải nằm trong thư mục `app/`. File `.dockerignore` ở repository root sẽ không được dùng cho build context này.

Build thử local nếu máy có Docker:

```
docker build -t demo-app:local ./app
docker run --rm -d --name demo-app-local -p 8082:8080 demo-app:local
curl http://localhost:8082
docker stop demo-app-local
```

Kỳ vọng: response chứa `Hello from CI Pipeline v1`.

Bước 2: Cập nhật Deployment

Trong `deploy/base/deployment.yaml`, đổi container spec thành:

```
      containers:
        - name: nginx
          image: <your-dockerhub-username>/demo-app:bootstrap
          ports:
            - name: http
              containerPort: 8080
          readinessProbe:
            httpGet:
              path: /
              port: http
            initialDelaySeconds: 2
            periodSeconds: 5
          livenessProbe:
            httpGet:
              path: /
              port: http
            initialDelaySeconds: 10
            periodSeconds: 10
          resources:
            requests:
              cpu: 50m
              memory: 32Mi
            limits:
              cpu: 200m
              memory: 128Mi
```

Tag `bootstrap` chỉ là giá trị khởi tạo. Workflow sẽ thay nó bằng short commit SHA ngay trong lần chạy đầu tiên.

Đổi `targetRevision` trong `argocd/application.yaml`:

```
targetRevision: part-2
```

Bước 3: Tạo GitHub secrets

3.1. Tạo Docker Hub access token

Không dùng password chính của Docker Hub cho CI. Tạo một Personal Access Token riêng:

1. Đăng nhập Docker Hub.
2. Chọn avatar ở góc trên bên phải → **Account settings**.
3. Chọn **Personal access tokens** → **Generate new token**.
4. Đặt tên dễ nhận biết, ví dụ `cmp-k8s-labs-github-actions`.
5. Chọn ngày hết hạn phù hợp với thời gian học.
6. Chọn quyền **Read & Write**. Workflow cần `Read` để làm việc với repository image và `Write` để push image mới; không cần quyền `Delete`.
7. Chọn **Generate**, copy token ngay và lưu tạm vào password manager. Docker Hub chỉ hiển thị giá trị token một lần.

> Token là credential nhạy cảm tương đương password. Không paste vào source code, YAML, ảnh chụp màn hình, log hay commit Git. Sau khi kết thúc khóa học, hãy revoke/delete token nếu không còn dùng.

Tài liệu chính thức: Docker Docs — Personal access tokens.

3.2. Lưu credential vào GitHub Actions secrets

Trong GitHub repository `cmp-k8s-labs`, vào **Settings** → **Secrets and variables** → **Actions** → **New repository secret**, rồi tạo:

- `DOCKER_USERNAME`: Docker ID/username của bạn, không phải email.
- `DOCKER_TOKEN`: Personal Access Token vừa tạo, có quyền Read & Write.

Sau khi lưu, GitHub chỉ hiển thị tên secret chứ không hiển thị lại giá trị. Workflow sử dụng chúng qua:

```
username: ${{ secrets.DOCKER_USERNAME }}
password: ${{ secrets.DOCKER_TOKEN }}
```

Không in secret ra log, không commit token vào repository và không dùng secret trong pull request từ fork không tin cậy.

Bước 4: Tạo workflow không bị lặp

`.github/workflows/ci.yaml`:

```yaml
name: Build image and update desired state

on:
  push:
    branches: [part-2]
    paths:
      - "app/**"
      - ".github/workflows/ci.yaml"
  workflow_dispatch:

permissions:
  contents: write

concurrency:
  group: part-2-image-update
  cancel-in-progress: false

jobs:
  build-and-update:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4
        with:
          ref: part-2
          fetch-depth: 0

      - name: Set immutable image tag
        id: vars
        shell: bash
        run: echo "tag=${GITHUB_SHA::12}" >> "$GITHUB_OUTPUT"

      - name: Log in to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKER_USERNAME }}
          password: ${{ secrets.DOCKER_TOKEN }}

      - name: Build and push image
        uses: docker/build-push-action@v6
        with:
          context: ./app
          push: true
          tags: ${{ secrets.DOCKER_USERNAME }}/demo-app:${{ steps.vars.outputs.tag }}
```

```
      - name: Update only the application image
        uses: mikefarah/yq@v4
        env:
          IMAGE: ${{ secrets.DOCKER_USERNAME }}/demo-app:${{ steps.vars.outputs.tag
}}
        with:
          cmd: yq -i '.spec.template.spec.containers[] |= select(.name ==
"nginx").image = strenv(IMAGE)' deploy/base/deployment.yaml

      - name: Commit desired state when changed
        shell: bash
        run: |
          if git diff --quiet -- deploy/base/deployment.yaml; then
            echo "Manifest already references this image; nothing to commit."
            exit 0
          fi
          git config user.email "github-actions[bot]@users.noreply.github.com"
          git config user.name "github-actions[bot]"
          git add deploy/base/deployment.yaml
          git commit -m "chore: deploy image ${{ steps.vars.outputs.tag }} [skip ci]"
          git push origin HEAD:part-2
```

Vì workflow chỉ trigger khi `app/**` hoặc chính workflow thay đổi, commit chỉ sửa `deploy/**` sẽ **không kích hoạt build mới**. `[skip ci]` chỉ là lớp bảo vệ/phần mô tả bổ sung; không nên dựa duy nhất vào nó.

> Để tăng bảo mật trong môi trường thật, pin third-party action bằng commit SHA đã review thay vì chỉ dùng major tag. Các major tag ở đây giữ lab dễ đọc; hãy kiểm tra release/commit trước khi triển khai production.

Bước 5: Push trạng thái khởi tạo

Trước khi commit, đảm bảo bạn đang ở branch `part-2` vì workflow chỉ chạy trên branch này:

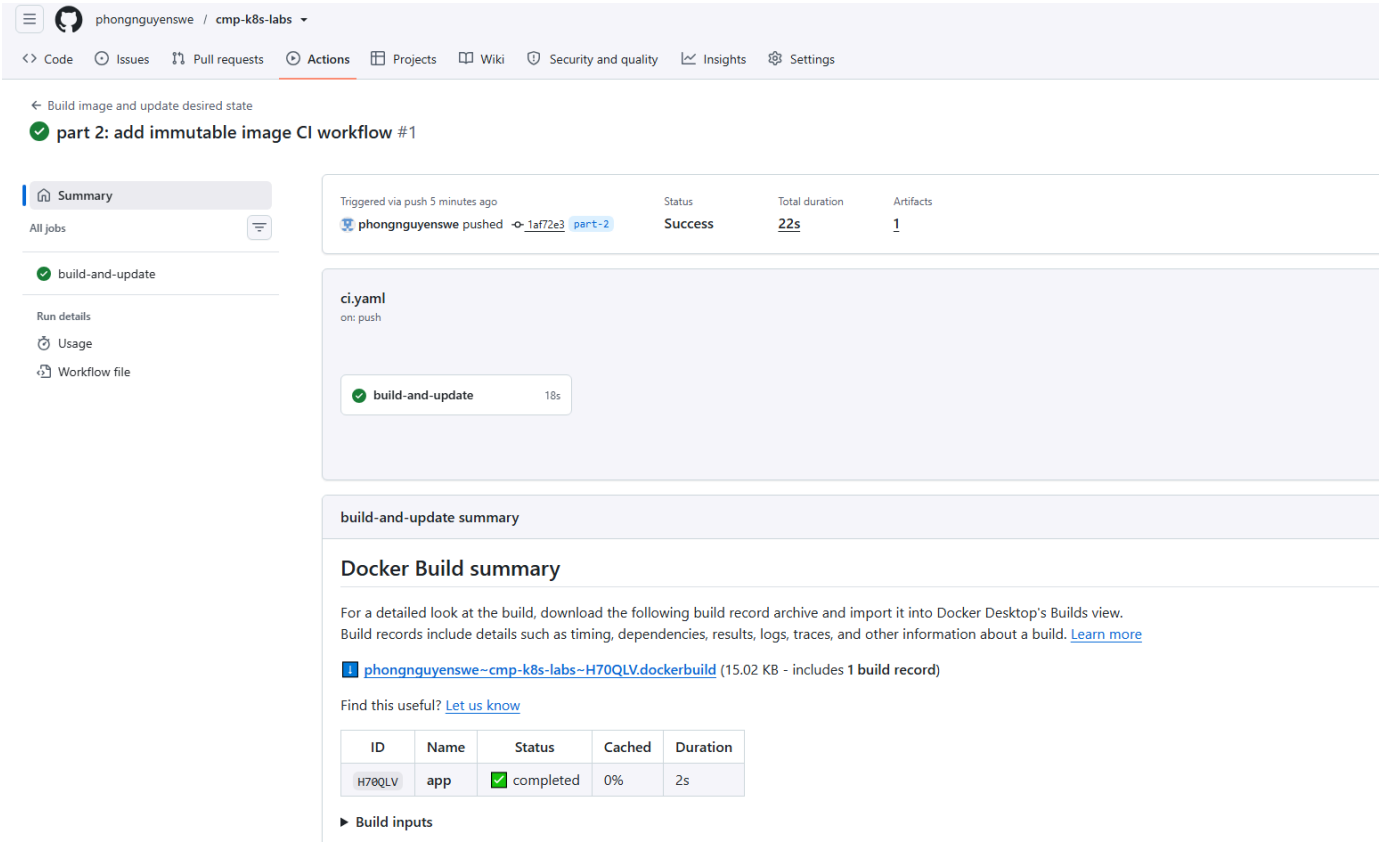
```
git branch --show-current
git checkout part-2
```

Kỳ vọng: branch hiện tại là `part-2`. Sau đó commit và push:

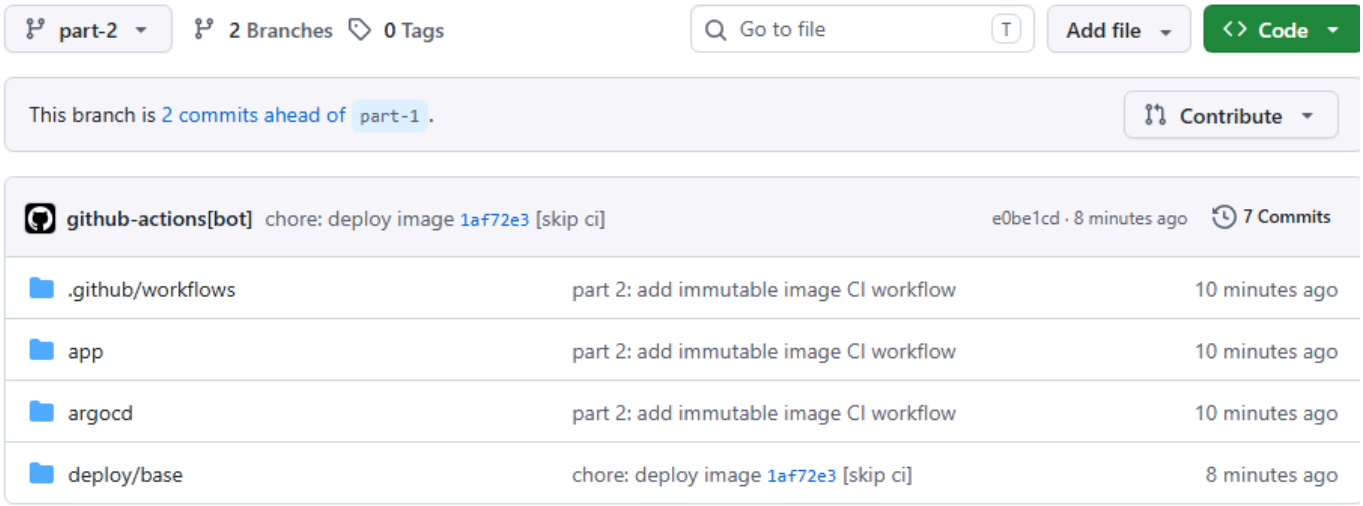
```
git add app deploy/base/deployment.yaml argocd/application.yaml
.github/workflows/ci.yaml
git commit -m "part 2: add immutable image CI workflow"
git push -u origin part-2
```

Theo dõi tab **Actions**. Một run thành công phải:

1. Build đúng một image tag bằng 12 ký tự đầu của commit SHA.
2. Push image lên Docker Hub.
3. Tạo đúng một commit cập nhật `deployment.yaml`.
4. Không sinh thêm workflow run từ commit manifest đó.



Kỳ vọng: workflow `Build image and update desired state` có trạng thái `Success` và job `build-and-update` hoàn tất. Chỉ có một workflow run được tạo từ commit khởi tạo trên branch `part-2`.



Trong lịch sử branch `part-2`, commit đầu tiên chứa source code và workflow; commit tiếp theo do `github-actions[bot]` tạo chỉ cập nhật `deploy/base/deployment.yaml` với image tag `1af72e3`. Commit bot không tạo thêm workflow run vì path filter chỉ theo dõi `app/**` và file workflow.

Nếu job không thể push, kiểm tra **Settings** → **Actions** → **General** → **Workflow permissions** và branch protection.

Workflow vừa tạo thêm một commit trên remote để cập nhật `deployment.yaml`. Đồng bộ commit đó về máy trước khi tiếp tục; nếu bỏ qua, lần `git push` sau có thể bị từ chối với lỗi `non-fast-forward`:

```
git pull --rebase origin part-2
git log --oneline --max-count=3
```

Kỳ vọng: commit mới nhất có dạng `chore: deploy image <short-sha> [skip ci]`. Kiểm tra manifest local đã chứa image tag mới, không còn `bootstrap`:

```
git grep 'image:' -- deploy/base/deployment.yaml
```

Bước 6: Chuyển Argo CD sang branch Part 2

Manifest trên cluster hiện vẫn theo cấu hình bootstrap cũ cho tới khi apply Application mới:

```
kubectl apply -f argocd/application.yaml
kubectl get application demo-app -n argocd -w
```

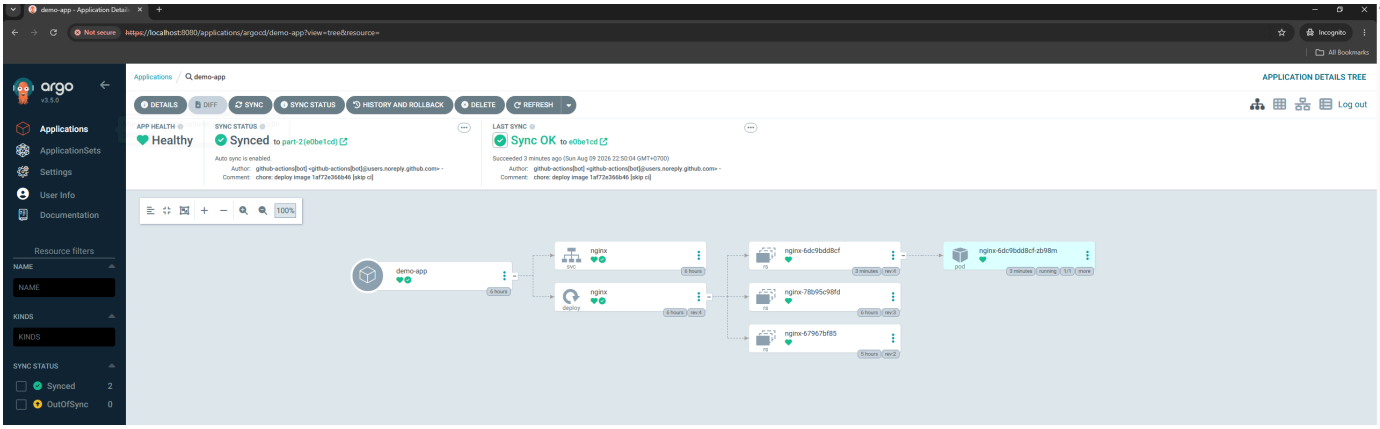
Dừng khi thấy `Synced/Healthy`, rồi kiểm tra:

```
kubectl rollout status deployment/nginx -n demo --timeout=180s
kubectl get deployment nginx -n demo -o
jsonpath='{.spec.template.spec.containers[0].image}';
kubectl get pods -n demo -o wide
```

```
PS F:\cmp-k8s-labs> kubectl apply -f argocd/application.yaml
application.argoproj.io/demo-app configured
PS F:\cmp-k8s-labs> kubectl get application demo-app -n argocd -w
NAME       SYNC STATUS   HEALTH STATUS
demo-app   Synced        Healthy
demo-app   Synced        Healthy
demo-app   OutOfSync     Healthy
demo-app   OutOfSync     Healthy
demo-app   OutOfSync     Healthy
demo-app   OutOfSync     Healthy
demo-app   OutOfSync     Healthy
demo-app   OutOfSync     Healthy
demo-app   OutOfSync     Healthy
demo-app   Synced        Progressing
demo-app   Synced        Progressing
demo-app   Synced        Healthy
demo-app   Synced        Healthy
PS F:\cmp-k8s-labs> kubectl rollout status deployment/nginx -n demo --timeout=180s
deployment "nginx" successfully rolled out
PS F:\cmp-k8s-labs> kubectl get deployment nginx -n demo -o jsonpath='{.spec.template.spec.containers[0].image}';
phongnguyenswe/demo-app:1af72e366b46
PS F:\cmp-k8s-labs> kubectl get pods -n demo -o wide
NAME                     READY   STATUS    RESTARTS   AGE   IP             NODE       NOMINATED NODE   READINESS GATES
nginx-6dc9bdd8cf-zb98m   1/1     Running   0          59s   10.244.120.81   minikube   <none>           <none>
PS F:\cmp-k8s-labs>
```

Trong ảnh, Application chuyển qua `OutOfSync`, sau đó `Synced/Progressing` và cuối cùng trở lại `Synced/Healthy`. Deployment rollout thành công, Pod mới ở trạng thái `Running` và image đang chạy có tag `1af72e366b46` — 12 ký tự đầu của source commit đã kích hoạt workflow.

Trên Argo CD UI, `demo-app` đã sync tới branch `part-2`; `Last Sync` trỏ tới commit do `github-actions[bot]` tạo và resource tree hiển thị Pod mới đang chạy:



Kiểm tra nội dung image đầu tiên do CI build:

```
kubectl port-forward service/nginx -n demo 8081:80
```

Mở `http://localhost:8081`. Kỳ vọng: trang hiển thị `Hello from CI Pipeline v1`.



Nhấn `Ctrl+C` để dừng port-forward trước khi tiếp tục.

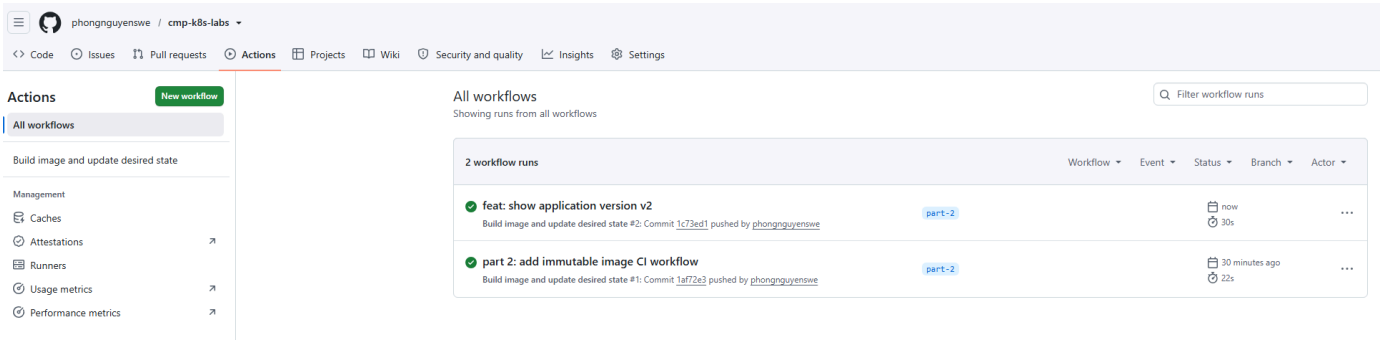
Bước 7: Kiểm chứng end-to-end

Ở cuối Bước 5, bạn đã chạy `git pull --rebase origin part-2` để lấy commit của bot về local. Vì vậy không cần pull lại ở đây.

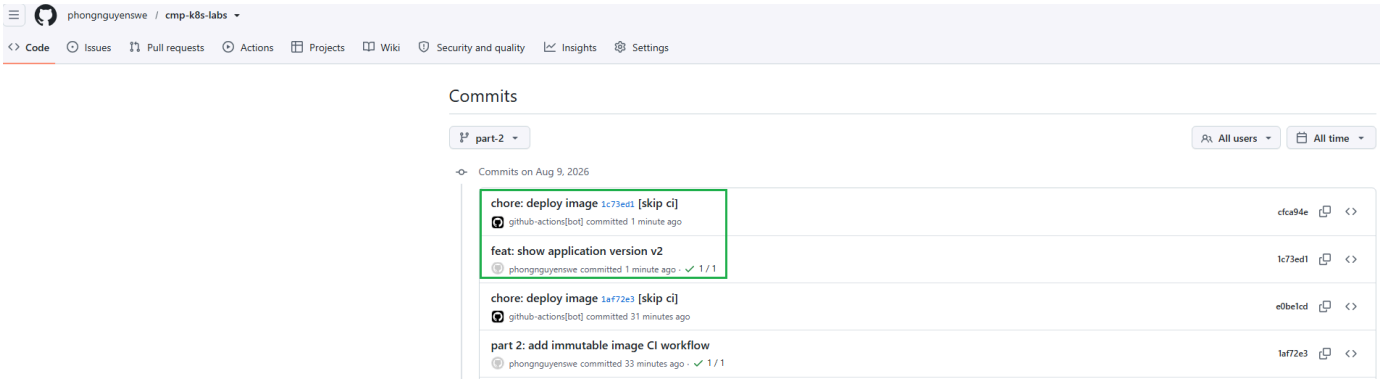
Đổi `v1` thành `v2` trong `app/index.html`, sau đó commit và push:

```
git add app/index.html
git commit -m "feat: show application version v2"
git push
```

> Đang đứng ở branch `part-2` không có nghĩa local tự động nhận commit mới trên remote. Nếu đã bỏ qua lệnh pull ở cuối Bước 5, hãy pull **trước khi sửa file**. Không pull với `--rebase` khi đang có unstaged changes.



Ảnh cho thấy có đúng hai workflow runs: lần đầu tạo pipeline và image v1, lần hai được kích hoạt bởi commit `feat: show application version v2`. Các commit do bot cập nhật manifest không tạo thêm run, vì vậy workflow không bị lặp.



Trong lịch sử Git, commit `feat: show application version v2` được theo sau bởi commit `chore: deploy image ... [skip ci]` của `github-actions[bot]`. Image tag trong bot commit tham chiếu tới SHA của source commit v2, giúp truy vết từ manifest đang deploy về đúng source code đã tạo ra image.

Sau khi workflow và Argo CD hoàn tất, chờ Deployment rollout:

```
kubectl rollout status deployment/nginx -n demo --timeout=180s
```

Nếu port-forward từ Bước 6 vẫn đang chạy, nhấn `Ctrl+C` ở terminal đó. Sau đó mở lại tunnel để chắc chắn kết nối tới Pod v2:

```
kubectl port-forward service/nginx -n demo 8081:80
```

> Không chạy hai lệnh port-forward cùng lúc trên port `8081`; tiến trình thứ hai sẽ báo lỗi `address already in use`.

Ở terminal khác:

```
curl http://localhost:8081
```

Kỳ vọng: response chứa `Hello from CI Pipeline v2`; image trong Deployment kết thúc bằng short SHA của commit source.

Sau khi workflow tạo manifest commit cho v2, đồng bộ local lần nữa:

```
git pull --rebase origin part-2
git log --oneline --max-count=3
```

Lúc này `HEAD` trong repository `cmp-k8s-labs` là manifest commit do bot tạo và `HEAD^` là source commit v2. Trên Windows PowerShell, chuyển tới repository bài học rồi chạy verifier:

```
cd F:\k8s-mentor\GitOps\series_blog

$env:EXPECTED_SOURCE_SHA = git -C F:\cmp-k8s-labs rev-parse HEAD^

& "C:\Program Files\Git\bin\bash.exe" scripts/verify-part-2.sh

Remove-Item Env:EXPECTED_SOURCE_SHA
```

Bước 8: Failure drill — image không tồn tại

Mục tiêu của bước này là mô phỏng lỗi khai báo một image tag không tồn tại, sau đó dùng Kubernetes để chẩn đoán và khôi phục qua Git.

Ở cuối Bước 7, `deploy/base/deployment.yaml` đang chứa image hợp lệ, ví dụ:

```
image: <your-dockerhub-username>/demo-app:1c73ed1...
```

Sửa **chỉ phần tag sau dấu** `:` thành:

```
image: <your-dockerhub-username>/demo-app:tag-does-not-exist
```

Commit và push thay đổi lỗi lên branch `part-2`:

```
git add deploy/base/deployment.yaml
git commit -m "failure drill: deploy a nonexistent image tag"
git push
```

Workflow GitHub Actions **không chạy** cho commit này vì chỉ có file trong `deploy/**` thay đổi, trong khi workflow chỉ theo dõi:

```
paths:
  - "app/**"
  - ".github/workflows/ci.yaml"
```

Argo CD vẫn phát hiện desired state mới trong Git và cập nhật Deployment. Theo dõi:

```
kubectl get application demo-app -n argocd -w
```

Ở terminal khác:

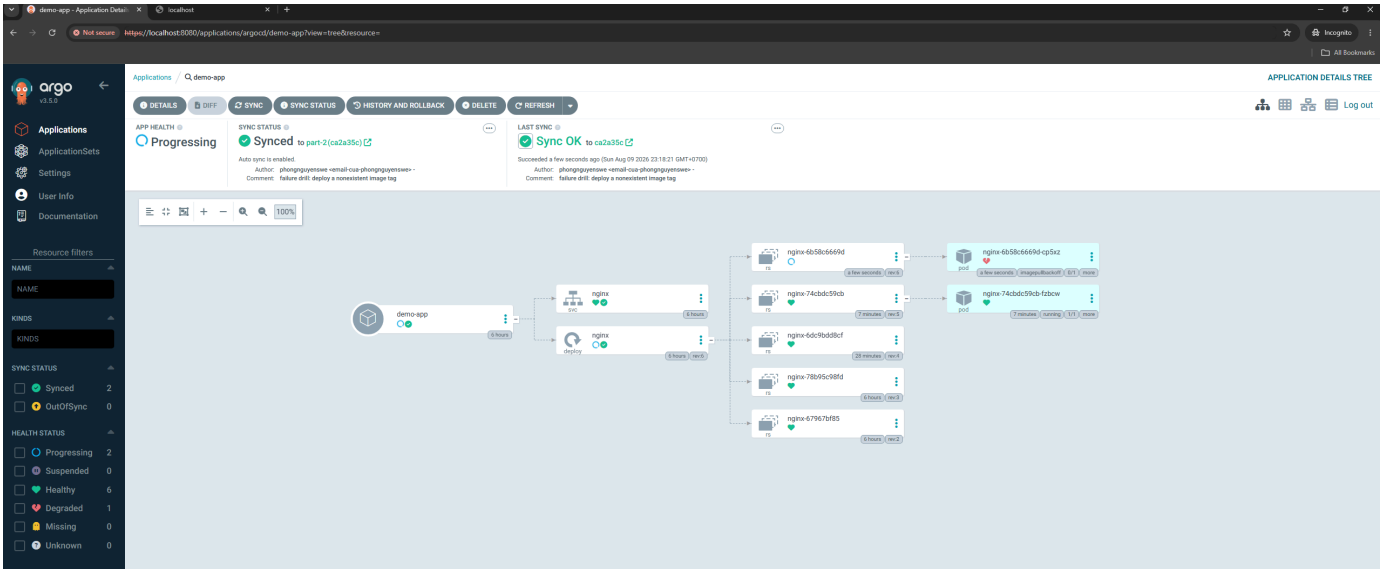
```
kubectl get pods -n demo -w
```

Kỳ vọng: Pod mới không thể pull image và chuyển sang `ErrImagePull` hoặc `ImagePullBackOff`. Pod cũ có thể vẫn `Running` trong lúc rolling update để hạn chế downtime.

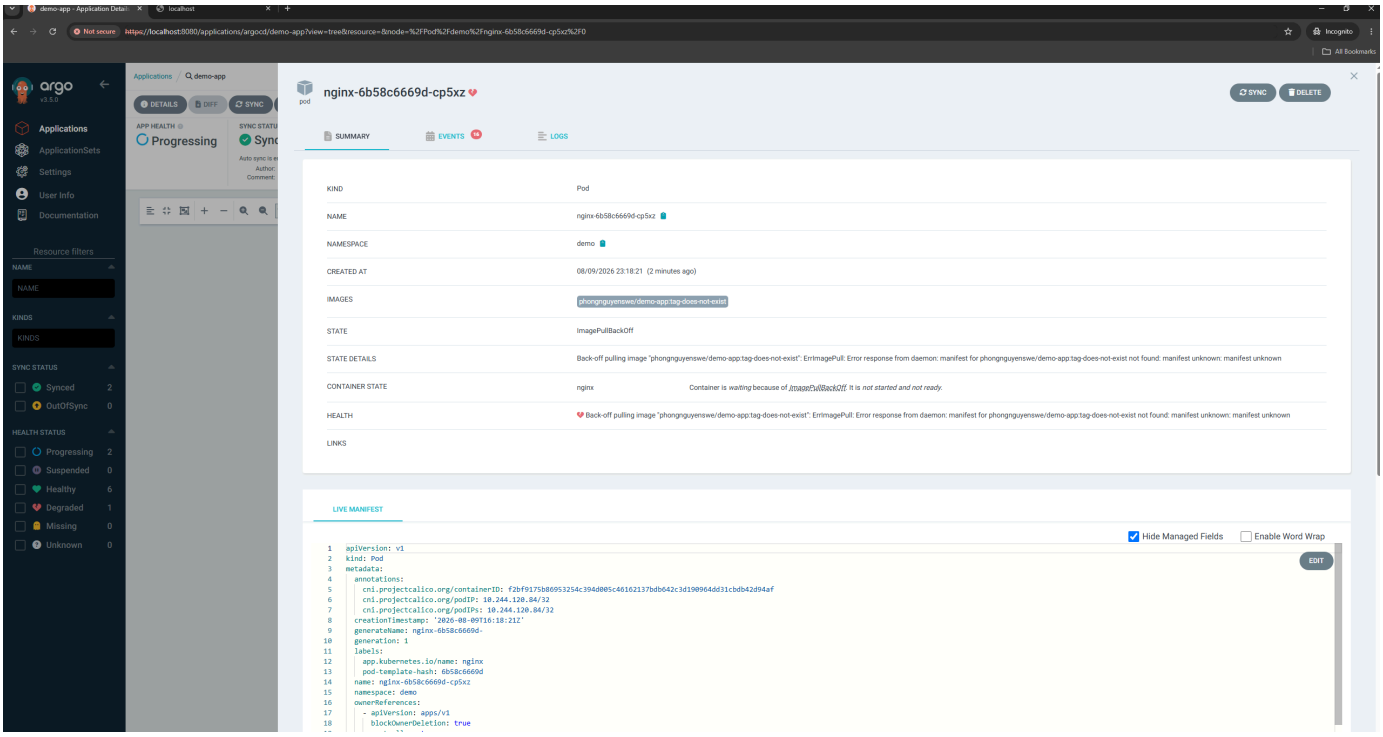
```
PS F:\k8s-mentor> kubectl get pods -n demo -w
NAME                     READY   STATUS              RESTARTS   AGE
nginx-74cbdc59cb-fzbcw   1/1     Running             0          5m36s
nginx-6b58c6669d-cp5xz   0/1     Pending             0          0s
nginx-6b58c6669d-cp5xz   0/1     Pending             0          0s
nginx-6b58c6669d-cp5xz   0/1     ContainerCreating   0          0s
nginx-6b58c6669d-cp5xz   0/1     ContainerCreating   0          1s
nginx-6b58c6669d-cp5xz   0/1     ErrImagePull        0          4s
nginx-6b58c6669d-cp5xz   0/1     ErrImagePull        0          4s
nginx-6b58c6669d-cp5xz   0/1     ErrImagePull        0          5s
nginx-6b58c6669d-cp5xz   0/1     ImagePullBackOff    0          5s
nginx-6b58c6669d-cp5xz   0/1     ErrImagePull        0          32s
```

Trong ảnh, Pod cũ vẫn `1/1 Running`, còn Pod mới lần lượt chuyển từ `Pending` → `ContainerCreating` → `ErrImagePull` / `ImagePullBackOff`. Điều này cho thấy Kubernetes không thể tải image mới và chưa thay thế Pod cũ đang khỏe mạnh.

Trên Argo CD UI, Application vẫn `Synced` vì live manifest đã giống Git, nhưng health status là `Progressing`; Pod mới được đánh dấu `ImagePullBackOff/Degraded`, trong khi Pod cũ vẫn khỏe mạnh:



Chọn Pod lỗi trên resource tree để xem chi tiết. Tab **Summary** hiển thị image `demo-app:tag-does-not-exist`, state `ImagePullBackOff` và thông báo `manifest unknown/not found`:



Thông báo này xác nhận registry truy cập được nhưng tag được khai báo không tồn tại. Đây cũng là thông tin bạn sẽ thấy khi chạy `kubectl describe pod`.

Điểm quan trọng: `Synced` chỉ cho biết cluster đã nhận đúng desired state từ Git; nó không đảm bảo workload mới chạy thành công. Cần đọc thêm health status và trạng thái từng resource.

Chẩn đoán:

```
kubectl get pods -n demo
kubectl describe pods -n demo
kubectl get events -n demo --sort-by=.lastTimestamp
```

Trong phần Events, tìm thông báo registry không tìm thấy manifest/tag `tag-does-not-exist`.

Khôi phục bằng cách revert commit vừa gây lỗi:

```
git revert --no-edit HEAD
git push
```

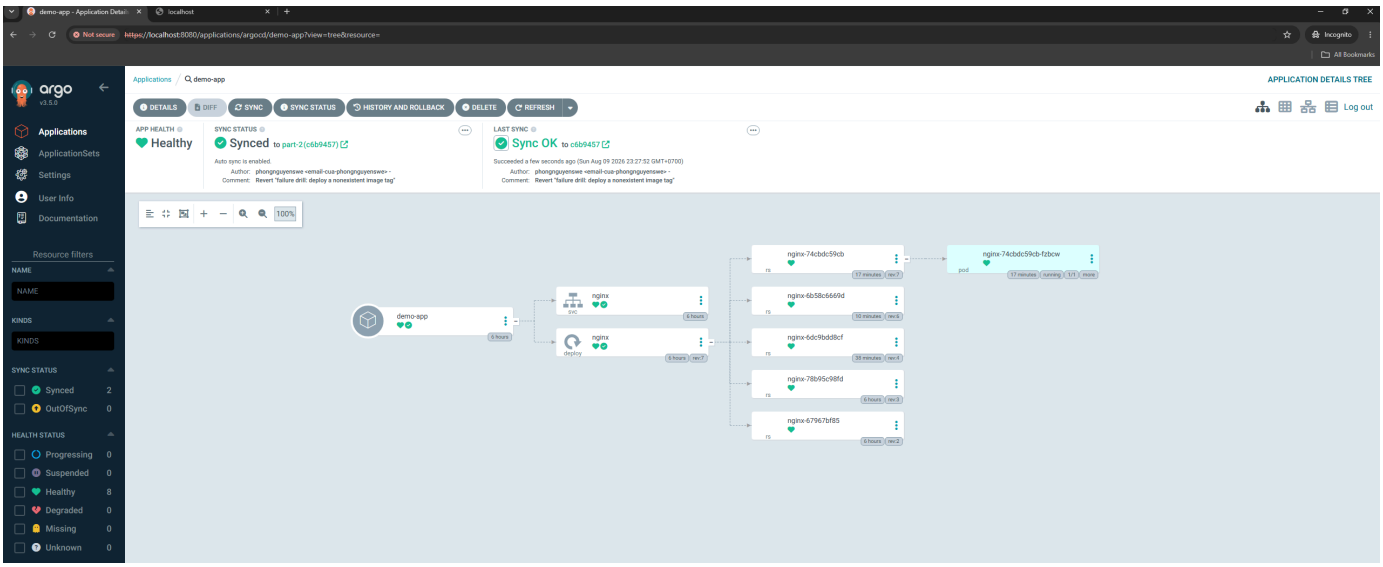
Chờ Argo CD áp lại image hợp lệ, rồi kiểm tra:

```
kubectl get application demo-app -n argocd -w
```

Nhấn `Ctrl+C` khi thấy `Synced/Healthy`, sau đó:

```
kubectl rollout status deployment/nginx -n demo --timeout=180s
kubectl get pods -n demo
kubectl get deployment nginx -n demo -o
jsonpath='{.spec.template.spec.containers[0].image}'; echo
```

Kỳ vọng: Pod lỗi biến mất, Pod dùng image hợp lệ là `Running` và Application trở lại `Synced/Healthy`.



Trong ảnh, Argo CD đã đồng bộ commit `Revert "failure drill: deploy a nonexistent image tag"`. Application trở lại `Synced/Healthy`, Deployment sử dụng lại image hợp lệ và Pod mới đã ở trạng thái `Running`.

6. Troubleshooting nhanh

Triệu chứng	Kiểm tra
Workflow liên tục tạo run	`paths` có đúng không; commit bot có sửa `app/**` không
`git push` trả 403	`permissions: contents: write`, workflow permissions, branch protection
Build không login registry	Tên secret, access token và quyền repository
Pod pull image lỗi	Tag trên Docker Hub, image public/private, `imagePullSecrets`
App vẫn hiện v1	Image Deployment, rollout, browser cache và response từ `curl`

7. Definition of done

Trên Windows PowerShell, chạy verifier từ repository bài học. Tùy chọn `git -C F:\cmp-k8s-labs` giúp lấy commit SHA từ repository lab mà không cần chuyển thư mục sang đó:

```
cd F:\k8s-mentor\GitOps\series_blog

$env:EXPECTED_SOURCE_SHA = git -C F:\cmp-k8s-labs rev-parse HEAD^

& "C:\Program Files\Git\bin\bash.exe" scripts/verify-part-2.sh

Remove-Item Env:EXPECTED_SOURCE_SHA
```

> Nếu lưu hai repository ở vị trí khác, hãy thay hai đường dẫn `F:\k8s-mentor\GitOps\series_blog` và `F:\cmp-k8s-labs` cho phù hợp.

- [ ] Một app commit tạo đúng một workflow run và một manifest commit.
- [ ] Không có vòng lặp workflow.
- [ ] Image tag khớp short SHA của source commit.
- [ ] Argo CD trở lại `Synced/Healthy`.
- [ ] `curl` trả nội dung v2.
- [ ] Bạn chẩn đoán và rollback được image tag lỗi bằng Git.

8. Cleanup

Nếu tiếp tục Part 3, giữ nguyên cluster. Xóa image test cũ trên Docker Hub bằng UI nếu không còn dùng; việc xóa registry artifact không được thực hiện tự động trong lab.

9. Challenge

Thêm một bước kiểm tra HTML vào workflow trước khi build. Workflow phải fail nếu không tìm thấy thẻ `<h1>` và tuyệt đối không push image khi test fail.

Sau khi hoàn thành hoặc hết thời gian, đối chiếu với lời giải Challenge Part 2.

10. Bài tiếp theo

Ở Part 2, chúng ta mới chỉ deploy vào một môi trường và sửa trực tiếp Kubernetes manifest. Khi dự án có thêm môi trường `dev` và `prod`, việc sao chép nhiều bộ YAML sẽ khó bảo trì và dễ làm cấu hình giữa các môi trường bị lệch nhau.

Trong Part 3, chúng ta sẽ giải quyết vấn đề đó bằng Helm và xây dựng quy trình promotion an toàn hơn:

```
Một Helm chart dùng chung
        ↓
values-dev.yaml và values-prod.yaml lưu cấu hình riêng
        ↓
CI build image mới và tự động cập nhật môi trường dev
        ↓
Kiểm tra phiên bản trên dev
        ↓
Tạo pull request để promotion đúng image đó sang prod
        ↓
Argo CD đồng bộ từng môi trường từ Git
```

Bạn cũng sẽ dùng `helm lint` và `helm template` để phát hiện lỗi chart trước khi deploy. Điểm quan trọng là prod không build lại image: cùng một artifact đã được kiểm chứng trên dev sẽ được promotion sang prod thông qua thay đổi được review trên Git.

Sau Part 3, bạn sẽ có một mô hình GitOps gần với dự án thực tế hơn: cấu hình nhiều môi trường rõ ràng, thay đổi production có lịch sử phê duyệt và mọi phiên bản đang chạy đều có thể truy vết từ Git.

Trong thời gian tới, team mình sẽ triển khai khóa học CKA, CKAD với nhiều lab thực chiến tương tự — bạn có thể tham khảo thêm tại: https://cloudmentor.pro/courses/cka

Chúc bạn học tập hiệu quả và sớm chinh phục được chứng chỉ CKA, CKAD! 💪